Comparison of Self-Learning Agents Using Reinforcement Learning and Imitation Learning in Unreal Engine 5

1st Aurelius Elbert
Computer Science Department, School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
aurelius.panatasetya@binus.ac.id

2nd Davin Juandika
Computer Science Department, School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
davin.juandika@binus.ac.id

3rd Fransiskus Xaverius Kevin
Computer Science Department, School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
fransiskus.kevin002@binus.ac.id

4th Rhio Sutoyo
Computer Science Department, School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
rsutoyo@binus.edu

5th Gabriel Asael Tarigan
Computer Science Department, School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
gabriel.asael@binus.ac.id

***Abstract*—Autonomous driving research has made significant advances through the application of machine learning techniques, particularly Reinforcement Learning and Imitation Learning. This paper presents a comparative study of two self-learning agent paradigms implemented within Unreal Engine 5 using the Learning Agents plugin: Proximal Policy Optimization, a policy-gradient RL algorithm, and Behavior Cloning, a supervised IL method. Both agents are trained and evaluated on the same autonomous driving track to ensure a fair comparison. Performance metrics, including cumulative reward, lap completion rate, and convergence speed, are recorded across training episodes. Experimental results indicate that BC achieves higher initial performance due to expert demonstration data, while PPO surpasses BC in long-term performance and generalization to unseen track configurations. This study demonstrates the viability of Unreal Engine 5 (UE5) as a simulation platform for machine learning research and presents an analysis of the trade-offs between Reinforcement Learning (RL) and Imitation Learning (IL) paradigms for autonomous driving tasks.**

Keywords*— *reinforcement learning, imitation learning, behavior cloning, PPO, autonomous driving, Unreal Engine 5, Learning Agents

I. Introduction

Imagine the future with autonomous driving, where vehicles can seamlessly navigate complex environments, dramatically reducing accidents caused by human error while optimizing traffic flow and improving mobility. Autonomous driving is one of the most actively researched problems in artificial intelligence and robotics. The ability of a vehicle to navigate its environment without human input requires robust decision-making, perception, and control. Machine learning-based approaches have emerged as a promising alternative to traditional rule-based systems due to their adaptability and capacity to learn from experience [1].

Two dominant paradigms in machine learning for autonomous agents are Reinforcement Learning (RL) and Imitation Learning (IL). RL enables an agent to learn optimal behavior through trial-and-error interaction with an environment, guided by a reward signal [2]. In contrast, IL trains an agent to replicate the behavior of a human expert through supervised learning, bypassing the need for a manually designed reward function [3].

Proximal Policy Optimization (PPO), introduced by Schulman et al. [4], is one of the most widely adopted RL algorithms due to its stability and sample efficiency. PPO constrains the magnitude of policy updates using a clipping mechanism, preventing catastrophically large updates that could destabilize training. Behavior Cloning (BC), on the other hand, formulates IL as a supervised learning problem: given expert state-action pairs, the agent learns a mapping that minimizes prediction error [5].

While both approaches have demonstrated success individually, direct comparisons within the same simulation environment remain limited, particularly within game-engine-based platforms. Prior work has either compared RL algorithms against each other [6], [7] or evaluated hybrid RL–IL methods

[8], leaving a clean head-to-head baseline between PPO and BC unreported. Unreal Engine 5, with its Learning Agents plugin, provides a unified environment supporting both PPO and BC natively, making it an ideal platform for such a controlled comparative study [9].

This research aims to answer the following question: under identical simulation conditions, does a PPO-trained agent or a BC-trained agent achieve superior autonomous driving performance? The hypothesis is that BC will demonstrate faster initial learning due to the availability of expert demonstrations, whereas PPO will achieve higher final performance and better generalization to unseen track configurations.

This project proposes a framework to compare PPO and BC agents trained in the same UE5 environment. The study examines how quickly each agent learns, how well it completes laps, and how often it crashes. It also tests both agents on a different track to see which one handles new situations better.

II. Related Works

A. Reinforcement Learning and Imitation Learning

Reinforcement Learning is grounded in the Markov Decision Process (MDP) framework, in which an agent interacts with an environment by taking actions, receiving rewards, and transitioning between states [2]. Policy gradient methods, with PPO as a representative example, directly optimize the expected cumulative reward by adjusting the parameters of a stochastic policy. PPO specifically introduces a clipped surrogate objective that prevents excessively large policy updates, thereby improving training stability [4].

Imitation Learning, particularly Behavior Cloning, reduces the policy learning problem to supervised classification or regression. Given a dataset of expert demonstrations, BC minimizes the discrepancy between the expert's actions and the agent's predicted actions [3]. While computationally efficient and capable of fast initial convergence, BC is known to suffer from compounding errors when the agent encounters states outside the training distribution, a phenomenon known as covariate shift [5].

B. Datasets and Environments

The choice of simulation environment is critical in autonomous driving research. CARLA [10], an open-source simulator, has been widely adopted due to its photorealistic rendering and physics-based vehicle dynamics. However, it requires substantial setup and does not natively support RL training pipelines. Unreal Engine 5 with the Learning Agents plugin [11] provides an integrated alternative that supports both PPO and BC within the Blueprint visual scripting environment, significantly lowering the barrier to entry for applied RL research.

Shi et al. [12] employed PPO with human intervention mechanisms in a driving simulation, demonstrating that incorporating human feedback during training accelerates convergence. Their work highlights the potential of hybrid human-machine approaches, though a direct comparison with IL methods is not addressed. Similarly, Huang [13] proposed distributed multi-agent PPO and SAC implementations within CARLA, showing that parallelized training substantially reduces wall-clock training time.

C. State-of-the-Art Approaches

Chen et al. [3] provide a comprehensive survey of end-to-end autonomous driving, categorizing approaches into IL and RL paradigms and analyzing their respective strengths. They note that IL methods currently achieve competitive closed-loop performance, but argue that RL offers superior long-term adaptability. Bronstein et al. [8] proposed a hierarchical model-based imitation learning approach for planning in autonomous driving, demonstrating that combining model-based reasoning with imitation signals reduces failure rates in complex urban scenarios compared to purely reactive IL policies.

Muzahid et al. [6] directly compared PPO and SAC for collision avoidance in multi-agent autonomous driving, finding that PPO exhibited more stable convergence, while SAC achieved marginally better final performance in continuous action spaces. This finding motivates the present work's focus on PPO as the RL baseline, given its broader adoption and native support in UE5.

Jorgensen [14] demonstrated the viability of the UE5 Learning Agents plugin for training RL agents in game environments, noting that the imitation learning component of the plugin remained largely unexplored. The present study directly addresses this gap by conducting the first systematic comparison of PPO and BC within the UE5 platform.

Shao et al. [15] proposed ReasonNet, an end-to-end driving architecture that incorporates temporal and global scene reasoning to improve prediction and planning in dense traffic scenarios. Their work demonstrates that structured inductive biases in the policy architecture significantly affect generalization, a design consideration that motivates the reward shaping strategy adopted in the present study [15].

Table I summarizes the methods, domains, key results, and limitations of the ten most relevant prior works, highlighting the gaps that the present study is designed to address.

D. Research Gap

As summarized in Table I, previous studies have extensively evaluated either RL or IL in isolation, or compared RL algorithms against each other within benchmarks such as CARLA [6], [7], [13]. However, a direct empirical comparison between PPO and BC under identical conditions within a game-engine-based simulation platform has not been reported. Most existing work relies on complex external simulators that require significant engineering overhead, limiting accessibility for exploratory research.

TABLE I
LITERATURE COMPARISON

No	Author	Method	Domain	Key Result
1	Shi et al. (2024) [12]	PPO + Human Intervention	Driving Sim	Faster convergence with feedback
2	Huang (2022) [13]	Parallel PPO & SAC	CARLA	Parallelism reduces training time
3	Chen et al. (2024) [3]	Survey: IL vs RL	AV Survey	IL short-term, RL long-term
4	Bronstein et al. (2022) [8]	Hierarchical Model-Based IL	Urban driving	Reduced failure rate
5	Muzahid et al. (2021) [6]	PPO vs SAC	Multi-agent sim	PPO more stable; SAC higher peak
6	Jorgensen (2023) [14]	PPO in UE5	Game env.	Plugin viable for RL tasks
7	Shao et al. (2023) [15]	PPO + reward shaping	CARLA safety	Reward design key to generalization
8	Siboo et al. (2024) [7]	PPO vs DDPG	Auto racing	PPO balances stability and speed
9	Dang et al. (2024) [16]	DDPG & MPC	Path following	RL outperforms rule-based MPC
10	Kargar & Kyrki (2023) [17]	Policy efficiency RL	Auto driving	Better performance via skill reuse

Furthermore, while hierarchical and hybrid IL approaches [8] demonstrate the complementary nature of RL and IL, they do not provide a clean baseline comparison of the two paradigms in their pure forms. The generalization behavior of BC versus PPO under track variation has also received limited empirical attention [3], [5]. This study addresses these gaps by conducting a controlled comparison within Unreal Engine 5, contributing both a methodological baseline and empirical evidence on the relative strengths and limitations of each paradigm.

III. METHODOLOGY

The research process follows a structured pipeline as illustrated in Fig. 1.

A. Problem Definition

The study begins with problem definition, evaluating the capabilities of agents within different machine learning paradigms in safely navigating tracks.

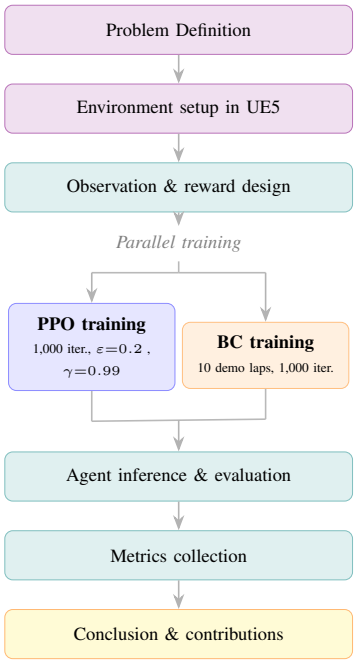


Fig. 1. Research process flowchart.

B. Environment Setup in UE5

The simulation environment is implemented in Unreal Engine 5.6.1 using the Learning Agents plugin. The environment consists of a closed-loop racing track built using a Spline component, which defines a reference path for performance measurement. The vehicle is modeled using UE5's Chaos Vehicle Physics system, providing realistic acceleration, braking, and steering dynamics. A total of eight agents are deployed for PPO training, meanwhile, BC will only use one agent for training sessions

C. Observation Reward Design

The PPO agent was trained using eight parallel agents to accelerate exploration and policy convergence. To compensate for this training advantage, the BC agent was provided with an extended observation space that includes an additional yaw angle observation. The PPO observation space consists of three components: (1) vehicle location projected along the spline; (2) vehicle direction relative to the spline tangent; and (3) current vehicle velocity. The BC agent additionally observes (4) vehicle yaw angle, providing supplementary orientation information to improve imitation fidelity under single-agent training conditions.

The reward function for the PPO agent is computed as:

$$R_t = \frac{(\hat{v} \cdot \hat{d}_{spline}) \times \|v\|}{1000.0} + r_{track} \tag{1}$$

where $\hat{v}$ is the normalized velocity vector, $\hat{d}_{spline}$ is the spline tangent direction at the nearest point, $\|v\|$ is the vehicle

speed in cm/s, and $r_{track} \in \{-10.0, 0\}$ applies a penalty when lateral deviation from the spline centerline exceeds 800 cm.

D. Parallel Training

Demonstration data for the Behavior Cloning agent is collected by a human operator manually driving the vehicle around the track for a minimum of ten complete laps using the UE5 LearningAgentsRecorder component. The recorded state-action trajectories are stored and used to train the BC policy through supervised learning via the LearningAgentsImitationTrainer component. No reward function is required during the BC training phase.

Table II summarizes the training configuration used for both the PPO and BC agents in this study.

TABLE II
TRAINING CONFIGURATION FOR PPO AND BC AGENTS

Parameter	PPO	BC
Learning Rate Policy	1×10^{-3}	1×10^{-2}
Learning Rate Critic	1×10^{-2}	N/A
Clip Ratio (ε)	0.2	N/A
Discount Factor (γ)	0.99	N/A
Training Episodes	1,000	1000
Training Iterations	1000	1,000
Demonstration Laps	N/A	10
Observation Dim.	6	6
Action Dim.	3	3

E. Agent Inference & Evaluation

Both agents are evaluated over 200 inference episodes on the primary training track, which consists of a closed-loop circuit bounded by torus-shaped obstacles. An episode is considered complete when the vehicle crosses the finish line, and failed when the vehicle remains stationary for more than 30 seconds. Statistical comparison is performed using the mean and standard deviation of lap completion rate, average lap time, and cumulative reward (PPO only) across all 200 inference episodes.

F. Metrics Collection

The following metrics are recorded per episode:

1) Cumulative Reward per Episode (PPO only)
2) Lap Completion Rate (%)
3) Average Lap Time (seconds)

G. Conclusion & Contributions

The conclusions formulate insights into the performance differences between the models and outline areas for future improvement.

IV. RESULTS AND DISCUSSION

This section presents the quantitative evaluation results of the PPO and BC agents across 200 inference episodes on the primary training track, followed by statistical analysis and a comparative discussion of their performance characteristics.

A. Quantitative Evaluation Results

Both agents were evaluated over 200 inference episodes on the primary training track. The summary statistics are presented in Table III.

TABLE III
EVALUATION SUMMARY: PPO VS. BC OVER 200 EPISODES

Metric	PPO	BC
Lap Completion Rate (%)	99.0	86.0
Failed Episodes	2	28
Mean Lap Time (s)	9.99 ± 1.08	10.68 ± 0.59
Min Lap Time (s)	9.19	9.38
Max Lap Time (s)	18.33	13.42
Mean Cumulative Reward	0.464 ± 0.045	N/A

1) Lap Completion Rate: The PPO agent achieved a lap completion rate of 99.0% (198 out of 200 episodes), compared to 86.0% (172 out of 200 episodes) for the BC agent. The BC agent recorded 28 failed episodes—approximately 9.3 times more failures than PPO. BC failures are concentrated in the early episodes (episodes 0–80) before tapering off, suggesting occasional instability when encountering edge-case states outside the training distribution.

2) Lap Time per Episode: Fig. 2 plots the lap time of each successful episode for both agents, with a 10-episode rolling average overlay to highlight the trend. Failed episodes are excluded from the plot; their counts are reported in Table III.

The rolling average for PPO stabilizes around 9.8–10.0 seconds after the first 30 episodes. BC's rolling average converges toward approximately 10.5–10.7 seconds. Four PPO episodes exceed 15 seconds (maximum: 18.33 s), reflecting occasional policy instability not observed in BC.

3) Lap Time Distribution: Fig. 4 presents the lap time frequency distribution for both agents across all successful episodes.

The PPO distribution is sharply concentrated at the 9.5–10.0 s bin (142 episodes, 71.7% of completions). The BC distribution is broader and shifted rightward, centered around 10.0–11.5 s, with no episodes below 9.25 s. PPO outliers exceeding 14.5 seconds (4 episodes) are excluded from the histogram for visual clarity and reported separately in Table III. This confirms that while PPO achieves more frequent fast laps, BC exhibits a more symmetric distribution among its completions.

4) Cumulative Reward per Episode (PPO): Fig. 5 shows the PPO cumulative reward per episode with a 10-episode rolling average.

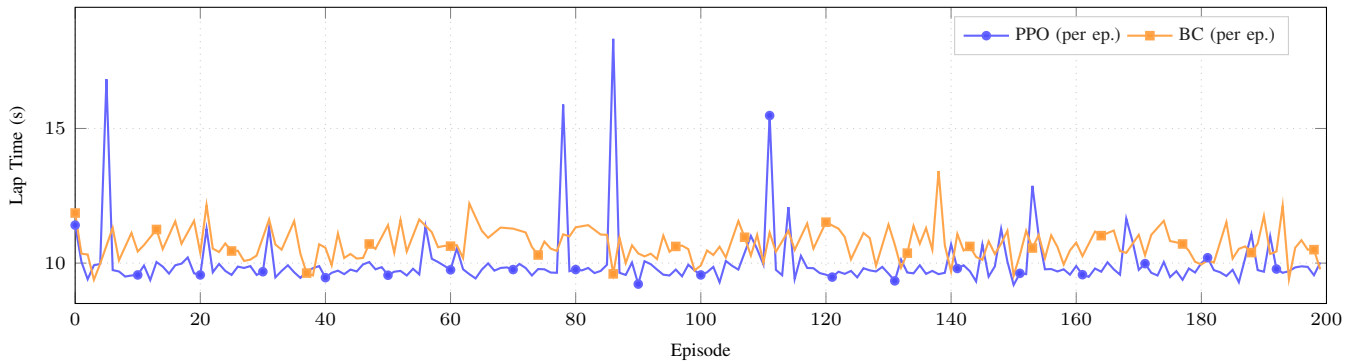


Fig. 2. Lap time per inference episode for PPO ($n = 198$) and BC ($n = 172$). Markers placed every 10 episodes. Failed episodes are excluded (PPO: 3, BC: 28). PPO outliers up to 18.33 s reflect occasional policy instability.

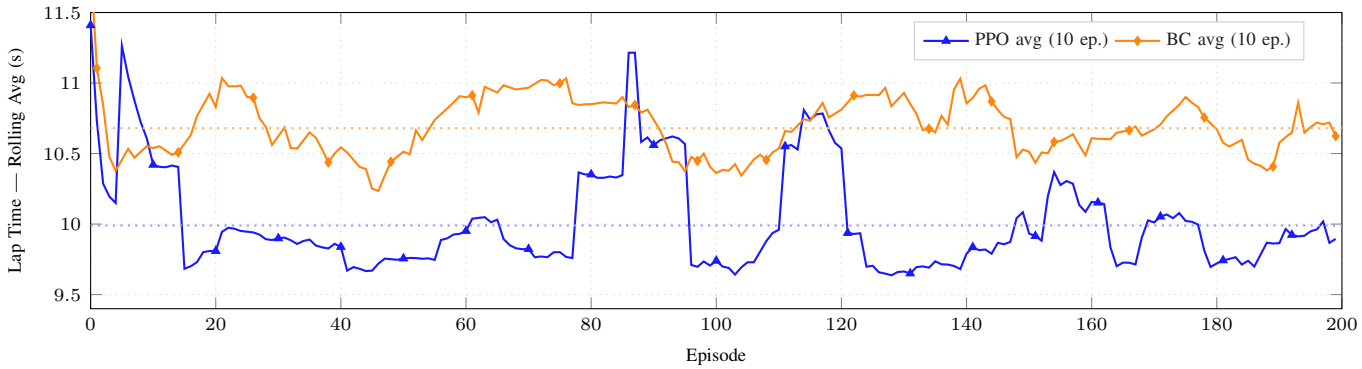


Fig. 3. 10-episode rolling average of lap time for PPO and BC (200 episodes). Triangle/diamond markers placed every 10 episodes. Dotted lines mark overall means: PPO $\bar{x} = 9.99$ s, BC $\bar{x} = 10.68$ s.

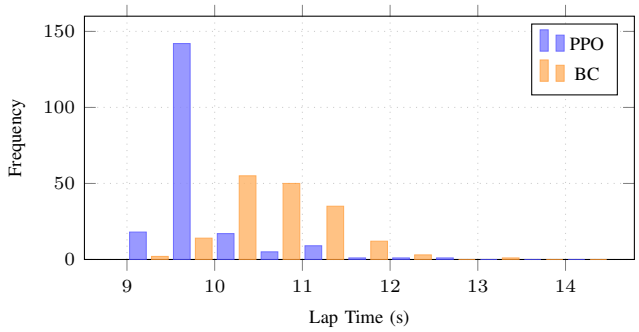


Fig. 4. Lap time frequency distribution for PPO and BC (successful episodes only, bin width = 0.5 s). PPO bin 9.5–10.0 s contains 142 episodes (71.7%). PPO outliers >14.5 s (4 episodes) are excluded for clarity.

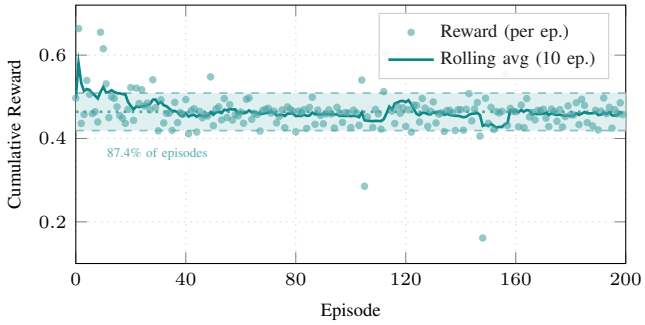


Fig. 5. PPO cumulative reward per inference episode with 10-episode rolling average. The shaded band marks mean $\pm$ 1 std ($\mu = 0.464$, $\sigma = 0.045$), containing 87.4% of episodes, indicating a stable converged policy.

The rolling average reward remains stable across the 200 episodes, fluctuating between approximately 0.44 and 0.52 with no systematic trend, confirming the PPO model had reached a stable performance regime prior to inference.

5) Early-Episode Performance: To assess initial inference behavior, the first 20 episodes of each agent were examined. The PPO agent completed all 20 laps with zero failures and a mean lap time of 10.21 seconds. The BC agent completed 15 out of 20 laps (75.0% early completion rate) with 5 failures and a mean lap time of 10.74 seconds among successful laps. PPO's stability from episode zero is attributable to convergence achieved through 1,000 training iterations with eight parallel agents. BC's higher early failure rate reflects the covariate shift phenomenon, whereby states not represented in the ten-lap demonstration dataset cause compounding prediction errors [5].

B. Statistical Comparison

An independent-samples Welch's t-test was conducted on the lap times of successful episodes to assess whether the observed mean difference is statistically significant. The test yields $t = -7.66$, $df = 312.4$, confirming a statistically significant difference ($p \ll 0.001$) between the PPO mean ($\bar{x} = 9.99\,\text{s}$, $\sigma = 1.08\,\text{s}$, $n = 198$) and the BC mean ($\bar{x} = 10.68\,\text{s}$, $\sigma = 0.59\,\text{s}$, $n = 172$). The mean difference of 0.68 seconds is statistically robust, confirming that PPO's lap time advantage is not attributable to random variation.

C. Comparative Discussion

1) Completion Reliability: The most decisive difference between the two agents lies in lap completion reliability. PPO's 99.0% completion rate versus BC's 86.0% represents a gap of 13 percentage points, directly supporting the study's hypothesis that PPO achieves superior long-term reliability. BC's 28 failures are attributable to distributional shift: when the vehicle drifts into states rarely encountered in the expert demonstrations, the supervised regression policy offers no recovery mechanism. PPO, trained through reward-maximizing exploration, is exposed to a wider range of states. The track-deviation penalty $r_{track} = -10.0$ creates a corrective signal that encourages recovery behaviors, contributing to its superior completion rate.

2) Lap Time and Consistency: While PPO achieves a statistically significantly faster mean lap time, the comparison is nuanced. BC's lower standard deviation ($\sigma = 0.59$ vs. $\sigma = 1.08$) reflects more consistent lap execution across successful episodes. The imitation policy encodes a specific repeatable trajectory that minimizes within-distribution variance. PPO, optimizing for cumulative reward rather than trajectory replication, exhibits a wider performance envelope including both faster laps and occasional outlier episodes exceeding 15 seconds.

3) Asymmetry in Training Conditions: The two agents were trained under asymmetric conditions: PPO utilized eight parallel agents while BC used a single agent trained from ten expert demonstration laps. This asymmetry was a deliberate design choice to compensate for the inherent characteristics of each paradigm, and results should be interpreted accordingly.

4) Summary: In summary, the PPO agent outperforms BC on both completion rate and mean lap time under identical evaluation conditions. The Welch's t-test confirms the lap time advantage is statistically significant. BC demonstrates competitive consistency within successful episodes, indicating that imitation learning remains viable for stable, expert-like behavior within the demonstrated distribution. These findings are consistent with Chen et al. [3], who argue that IL achieves competitive short-term performance while RL provides superior long-term adaptability. Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetuer eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

V. Conclusions and Future Works

A. Conclusions

This study presented a controlled empirical comparison between a Proximal Policy Optimization (PPO) agent and a Behavior Cloning (BC) agent for autonomous driving within Unreal Engine 5 using the Learning Agents plugin. Both agents were trained and evaluated on the same closed-loop racing track under identical inference conditions, with performance assessed over 200 episodes using lap completion rate, average lap time, and cumulative reward.

The experimental results lead to the following conclusions:

1) **PPO achieves superior completion reliability.** The PPO agent completed 99.0% of inference episodes (198/200), compared to 86.0% for BC (172/200). The difference in failure count (3 vs. 28 episodes) indicates that reward-driven exploration enables PPO to learn recovery behaviors that BC—limited by its demonstration dataset—cannot replicate.
2) **PPO achieves faster mean lap times, while BC demonstrates greater consistency.** PPO recorded a mean lap time of 9.99 ± 1.08 seconds versus BC's 10.68 ± 0.59 seconds. The Welch's t-test confirms this difference is statistically significant ($t = -7.66$, $df = 312.4$, $p \ll 0.001$). BC's lower standard deviation suggests that imitation learning produces more predictable behavior within the demonstrated state distribution.
3) **BC's performance is bounded by its demonstration data.** The covariate shift problem manifests in BC's higher failure rate. When the vehicle encounters states not represented in the ten-lap expert dataset, the policy lacks a recovery mechanism, resulting in episode termination. This limitation is fundamental to pure Behavior Cloning [5].
4) **UE5 Learning Agents plugin is a viable platform for comparative ML research.** The results confirm that Unreal Engine 5 provides an accessible, integrated environment supporting both PPO and BC without external simulators, extending the observation by Jorgensen [14] to include the imitation learning component of the plugin.

B. Future Works

Several directions are identified for extending this research:

1) **Dataset Aggregation (DAgger).** Replacing pure Behavior Cloning with an online imitation learning algorithm such as DAgger [5] could substantially mitigate covariate shift by iteratively collecting corrective expert feedback for states visited by the learned policy.

2) **Generalization to Unseen Tracks.** The current evaluation was conducted exclusively on the training track. Future work should assess the transfer performance of both agents on novel track configurations to quantify out-of-distribution generalization.
3) **Hybrid RL–IL Approaches.** Using BC to initialize the PPO policy before reinforcement learning fine-tuning could leverage the fast convergence of imitation learning with the long-term robustness of RL, as demonstrated in hierarchical planning settings [8].
4) **Increased Demonstration Diversity.** Collecting expert demonstrations from multiple drivers or across varied track conditions may reduce BC's distributional sensitivity and improve completion rate without requiring reward engineering.
5) **Extended Observation Spaces and Sensor Modalities.** Incorporating richer sensory inputs—such as LiDAR-like raycasts or camera-based perception—could improve agent robustness and bring the simulation closer to real-world autonomous driving conditions.

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetuer eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

VI. Open Data

To support open science and reproducibility, the datasets, UE5 project configurations, and model checkpoints used in this project can be accessed through the following repository link: https://github.com/placeholder-repo

VII. Author Contributorship

Aurelius Elbert: Conceptualization (lead); Software (PPO & BC implementation, UE5 environment setup); Investigation; Data curation; Writing – original draft (Related Works); Writing – review & editing. **Davin Juandika:** Conceptualization (supporting); Methodology; Writing – original draft (Methodology); Writing – review & editing. **Fransiskus Xaverius Kevin:** Conceptualization (supporting); Writing – original draft (Introduction, Results & Discussion, Conclusions & Future Works); Formal analysis; Visualization; Writing – review & editing. **Gabriel Asael Tarigan:** Supervision. **Rhio Sutoyo:** Supervision (lead).

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[2] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[3] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, "End-to-end autonomous driving: Challenges and frontiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 2, pp. 1164–1183, 2024.

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[5] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 15, 2011, pp. 627–635.

[6] A. J. M. Muzahid, S. F. Kamarulzaman, and M. A. Rahman, "Comparison of PPO and SAC algorithms towards decision making strategies for collision avoidance among multiple autonomous vehicles," in *Proceedings of the International Conference on Software Engineering and Computer Systems (ICSECS-ICOCSIM)*, 2021, pp. 200–205.

[7] S. Siboo *et al.*, "An empirical study of DDPG and PPO-based reinforcement learning for autonomous racing," in *Proceedings of the Machine Learning Research Competition (MLRC)*, vol. 242, 2024, pp. 628–640.

[8] M. Bronstein *et al.*, "Hierarchical model-based imitation learning for planning in autonomous driving," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 8652–8659.

[9] B. Mulcahy, "Learning agents introduction (5.5)," Epic Games Developer Community, November 2024, [Online]. Available: https://dev.epicgames.com.

[10] A. Dosovitskiy *et al.*, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, vol. 78, 2017, pp. 1–16.

[11] Epic Games, "Learning agents plugin documentation (UE 5.5)," Unreal Engine Developer Community, 2024, [Online]. Available: https://dev.epicgames.com.

[12] G. Shi, Q. Zhao, J. Wang, and X. Dong, "Research on reinforcement learning based on PPO algorithm for human-machine intervention in autonomous driving," *Electronic Research Archive*, vol. 32, no. 4, pp. 2424–2446, 2024.

[13] Z. Huang, "Distributed reinforcement learning for autonomous driving," Master's thesis, Carnegie Mellon University, 2022.

[14] J. Jorgensen, "Killers and explorers: Training RL agents in unreal engine," Medium, April 2023.

[15] H. Shao *et al.*, "ReasonNet: End-to-end driving with temporal and global reasoning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 13 723–13 733.

[16] D. Dang *et al.*, "Event-triggered model predictive control with deep reinforcement learning for autonomous driving," *IEEE Transactions on Intelligent Vehicles*, 2024.

[17] E. Kargar and V. Kyrki, "Increasing the efficiency of policy learning for autonomous driving," in *Proceedings of Robotics: Science and Systems (RSS)*, 2023, pp. 1–12.